1. Iterator

Real-Life Example

Imagine a book.

- You read **one page at a time**.
- You don't jump to all pages together.

Python Iterator also gives **one value at a time**.

Example

```
numbers = [10, 20, 30]

it = iter(numbers)   # Create iterator

print(next(it))
print(next(it))
print(next(it))
```

Output

```
10
20
30
```

How it works

```
numbers = [10, 20, 30]
```

Memory:

```
[10, 20, 30]
 ^
```

`next(it)` → 10

```
[10, 20, 30]
     ^
```

`next(it)` → 20

```
[10, 20, 30]
         ^
```

`next(it)` → 30

After that:

```
next(it)
```

Error:

```
StopIteration
```

---

2. Generator

Problem

Suppose we need numbers from 1 to 1,00,000.

Using list:

```
nums = [x for x in range(100000)]
```

All numbers are stored in memory.

Solution: Generator

Generator creates values **one by one**.

Example

```
def count():
    yield 1
    yield 2
    yield 3

g = count()

print(next(g))
print(next(g))
print(next(g))
```

Output

```
1
2
3
```

Difference

Normal Function:

```
def test():
    return 1
    return 2
```

Returns once and stops.

Generator:

```
def test():
    yield 1
    yield 2
```

Pauses and continues later.

Fibonacci Generator

```
def fibonacci():
    a, b = 0, 1

    while True:
        yield a
        a, b = b, a + b

fib = fibonacci()

for i in range(10):
    print(next(fib))
```

Output:

```
0
1
1
2
3
5
8
13
21
34
```

Iterator vs Generator

Iterator	Generator
Uses `iter()`	Uses `yield`
More code	Less code
Manually create class	Easy to create
Gives one value at a time	Gives one value at a time

---

3. Decorator

Real-Life Example

Suppose you buy a pizza.

Original Pizza:

```
Pizza
```

Add Cheese:

```
Pizza + Cheese
```

Add Cheese + Sauce:

```
Pizza + Cheese + Sauce
```

Decorator adds extra functionality to a function **without changing original code**.

Normal Function

```
def greet():
    print("Hello")
```

Output:

```
Hello
```

Decorator Example

```
def decorator(func):

    def wrapper():
        print("Before Function")

        func()

        print("After Function")

    return wrapper


@decorator
def greet():
    print("Hello")


greet()
```

Output

```
Before Function
Hello
After Function
```

What Happened?

Python changes:

```
@decorator
def greet():
    print("Hello")
```

into:

```
greet = decorator(greet)
```

So when:

```
greet()
```

runs:

```
Before Function
Hello
After Function
```

Easy Memory Trick

Iterator

👉 "Next value aapo"

```
next(it)
```

Generator

👉 "Yield kari ne value aapo"

```
yield value
```

Decorator

👉 "Function ma extra feature add karo"

```
@decorator
```

One-Line Definitions (Exam Point of View)

Iterator:
An iterator is an object that returns one element at a time using `next()`.

Generator:
A generator is a special function that uses `yield` to produce values one at a time.

Decorator:
A decorator is a function that adds extra functionality to another function without modifying its original code.

Think of it like this:

```
Iterator  → next()
Generator → yield
Decorator → @
```

This is the easiest way to remember all three. 🚀

Link : CHATGPT

Link : ALL